UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

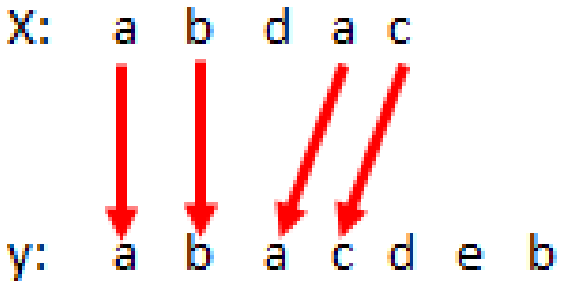
Temas a desarrollar

1. Repaso Clase Anterior
2. Resolución Ejercicios
3. Algoritmo de Floyd
4. Ejercicio sobre Floyd

Subsecuencia común más larga

Subsecuencia común más larga - Estrategia

La solución a este problema estará dada por considerar soluciones parciales en las que se consideran los primeros *i* caracteres de la secuencia *X* y los primeros *j* caracteres de la secuencia *Y*, generando la siguiente función:



$$L[i,j] \in \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ L[i-1, j-1] + 1 & \text{si } i > 0 \text{ y } j > 0 \text{ y } X_i = Y_j \\ \text{máx}(L[i-1,j], L[i,j-1]) & \text{sino} \end{cases}$$

Complejidad temporal O(X*Y)

UADE

Ejercicio a resolver

1. Preparación física de la selección

Luego de la consagración en Qatar 2022, la Selección Argentina se prepara para afrontar un nuevo Mundial con el sueño de alcanzar la tan ansiada cuarta estrella. Durante la concentración previa al debut, Lionel Scaloni y su cuerpo técnico deben decidir qué actividades de entrenamiento realizar para llegar en las mejores condiciones posibles al inicio de la competencia.

Los preparadores físicos han diseñado una lista de actividades de entrenamiento. Para cada actividad se conoce:

- la mejora de rendimiento r_i que aporta al equipo;
- la carga física c_i que genera sobre los jugadores.

De cada entrenamiento podemos realizar como máximo 2 sesiones. Además, los preparadores físicos han observado que, al repetir un mismo entrenamiento, los jugadores se adaptan mejor a la exigencia física. Por este motivo, la segunda sesión de un entrenamiento genera un 20% menos de carga física que la original y aporta un 10% más de la mejora de rendimiento. Por otro lado, se tiene definido la carga máxima que se puede exigir y la misma no puede ser superada.

Diseñe un algoritmo que calcule la mejora total máxima del rendimiento del equipo sin superar la carga física máxima permitida.

Algoritmo de Floyd

Algoritmo de Floyd

- El algoritmo de Floyd consiste en la **búsqueda del costo de caminos mínimos** entre cualquier par de nodos de un grafo dirigido, siempre que existan.

- **A diferencia del algoritmo de Dijkstra**, este funciona solo con un grafo y no necesita un vértice particular.

- **Al igual que el algoritmo de Dijkstra,** el resultado es un grafo con los mismos nodos que el grafo de entrada, pero tiene una arista entre cada par de nodos y su peso es el costo de un camino mínimo.

- **Para poder ejecutar el algoritmo** es necesario contar con un grafo dirigido, con todas sus aristas con peso positivo.

Algoritmo de Floyd - Estrategia

- Si planteamos una solución en la que buscamos el **costo entre cada par de nodos** contemplando pasar por todos los demás y lo planteamos como lo mejor de no contemplar uno de los nodos y luego probar agregarlo, podremos llegar a una solución óptima basado en soluciones menores también óptimas.

- Es decir, si tenemos la mejor forma de llegar de cada nodo a cualquier otro, pasando por el subconjunto de los vértices como nodos intermedios y luego contemplamos pasar además por un vértice *k*, podríamos refinar cada resultado parcial y ver si utilizando el nodo *k* se puede mejorar.

Algoritmo de Floyd - Estrategia

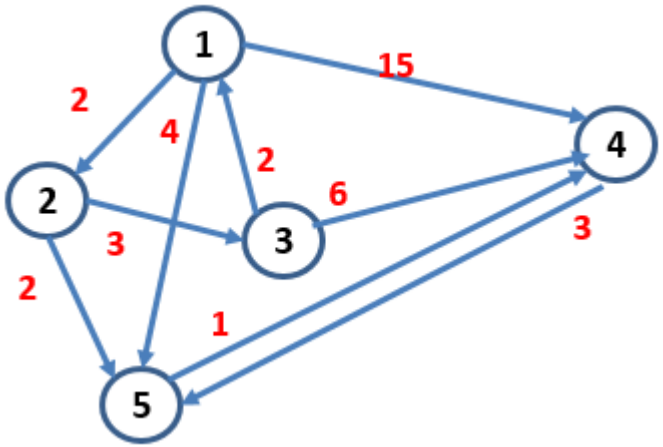
- Partiremos de una matriz M_0 en donde tenemos la mejor forma de llegar de cada vértice a todos los demás sin pasar por ningún vértice intermedio e iremos encontrando las matrices intermedias M_1, M_2 hasta llegar a M_n donde n es la cantidad de vértices del grafo.

- M_0 estará inicializado con *0* en su diagonal, el valor de la arista en el grafo original cuando exista arista o infinito si no existe arista.

- Luego M_k se calculará cómo sigue:

$$M_k[i,j] = \text{mín}(M_{k-1}[i,j], M_{k-1}[i,k] + M_{k-1}[k,j])$$

Algoritmo de Floyd - Ejemplo



Se inicializa la matriz resultado, que será el grafo resultante luego de ejecutar todo el algoritmo de Floyd.

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	∞	0	6	∞
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Algoritmo de Floyd - Ejemplo

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	∞	0	6	∞
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Se va a mostrar la iteración para *k=1* (lo mismo se debe seguir para el resto).

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	∞	0	6	∞
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

i=1	j=2	peso(1,1) + peso(1,2) ? Peso(1,2)	Como la arista es la misma en la primer parte que la segunda no cambia
	j=3	peso(1,1) + peso(1,3) ? Peso(1,3)	
	j=4	peso(1,1) + peso(1,4) ? Peso(1,4)	
	j=5	peso(1,1) + peso(1,5) ? Peso(1,5)	

Algoritmo de Floyd - Ejemplo

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	∞	0	6	∞
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Se continúa trabajando con *k=1* para los pares de vértices que comienzan con 2.

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	∞	0	6	∞
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

i=2	j=1	peso(2,1) + peso(1,1) ? Peso(2,1)	Como peso (2,1) es infinito no cambia
	j=3	peso(2,1) + peso(1,3) ? Peso(2,3)	
	j=4	peso(2,1) + peso(1,4) ? Peso(2,4)	
	j=5	peso(2,1) + peso(1,5) ? Peso(2,5)	

Algoritmo de Floyd - Ejemplo

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	∞	0	6	∞
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Se continúa trabajando con *k=1* para los pares de vértices que comienzan con 3

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	4	0	6	6
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

i=3	j=1	peso(3,1) + peso(1,1) ? Peso(3,1)
	j=2	peso(3,1)=2 + peso(1,2)=2 ? Peso(3,2)=∞
	j=4	peso(3,1)=2 + peso(1,4)=15 ? Peso(3,4)=6
	j=5	peso(3,1) =2+ peso(1,5)=4 ? Peso(3,5)=∞

Algoritmo de Floyd - Ejemplo

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	4	0	6	6
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Se continúa trabajando con *k=1* para los pares de vértices que comienzan con 4

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	4	0	6	6
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

i=4	j=1	peso(4,1) + peso(1,1) ? Peso(4,1)	Dado que el peso es infinito de (4,1)
	j=2	peso(4,1) + peso(1,2) ? Peso(4,2)	no se obtiene mejor resultado
	j=3	peso(4,1) + peso(1,3) ? Peso(4,3)	
	j=5	peso(4,1) + peso(1,5) ? Peso(4,5)	

Algoritmo de Floyd - Ejemplo

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	4	0	6	6
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Se continúa trabajando con *k=1* para los pares de vértices que comienzan con 5

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	4	0	6	6
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

Se debe continuar el proceso con k=2,3,4,5 para llegar al resultado final

i=5	j=1	peso(5,1) + peso(1,1) ? Peso(5,1)	Dado que el peso es infinito de (5,1)
	j=2	peso(5,1) + peso(1,2) ? Peso(5,2)	no se obtiene mejor resultado
	j=3	peso(5,1) + peso(1,3) ? Peso(5,3)	
	j=4	peso(5,1) + peso(1,4) ? Peso(5,4)	

Algoritmo de Floyd - Ejemplo

Luego de haber analizado todos los pares de nodos pasando por el nodo *k=1* se llegó al siguiente resultado:

	1	2	3	4	5
1	0	2	∞	15	4
2	∞	0	3	∞	2
3	2	4	0	6	6
4	∞	∞	∞	0	3
5	∞	∞	∞	1	0

El proceso realizado anteriormente se debe repetir para el resto de los nodos *k*, es decir para 2, 3, 4 y 5, para nuevamente todo par *(i,j)* del grafo.

Algoritmo de Floyd - Algoritmo

ALGORITMO FLOYD

Entrada: G grafo de entrada

Salida: R: grafo con una arista con la distancia mínima entre cada par de vértices

$inicializarGrafo(R)$

$copiarGrafo(G, R)$

$Vertices_K = Vertices(G)$

Continúa en la próxima *slide*

Algoritmo de Floyd - Algoritmo

```
para cada k ∈ Vertices_K
  Vertices_I = Vertices(G)
  para cada i ∈ Vertices_I
    Vertices_J = Vertices(G)
    para cada j ∈ Vertices_J
      si i ≠ j Y existeArista(R,i,k) Y existeArista(R,k,j)
        si existeArista(R,i,j)
          si pesoArista(R,i,k) + pesoArista(R,k,j) <
          pesoArista(R,i,j)
            agregarArista(R,i,j,pesoArista(R,i,k) +
            pesoArista(R,k,j))
          fin si
        sino
          agregarArista(R,i,j,pesoArista(R,i,k)+pesoArista(R,k,j))
        fin si
      fin si
    fin para
  fin para
fin para
devolver R
```

Algoritmo de Floyd – Complejidad Temporal

ALGORITMO FLOYD

Entrada: G grafo de entrada

Salida: R: grafo con una arista con la distancia mínima entre cada par de vértices

$inicializarGrafo(R)$

$copiarGrafo(G, R)$

$Vertices_K = Vertices(G)$

Continúa en la próxima *slide*

Algoritmo de Floyd – Complejidad Temporal

```
para cada k ∈ Vertices_K
    Vertices_I = Vertices(G)
    para cada i ∈ Vertices_I
        Vertices_J = Vertices(G)
        para cada j ∈ Vertices_I
            si i ≠ j Y existeArista(R,i,k) Y existeArista(R,k,j)
                si existeArista(R,i,j)
                    si pesoArista(R,i,k) + pesoArista(R,k,j) <
                    pesoArista(R,i,j)
                        agregarArista(R,i,j,pesoArista(R,i,k) +
                        pesoArista(R,k,j))
                    fin si
                sino
                    agregarArista(R,i,j,pesoArista(R,i,k)+pesoArista(R,k,j))
                fin si
            fin si
        fin para
    fin para
fin para
devolver R
```

$\mathcal{O}(n^3)$

Aplicación de Algoritmo de Floyd

UADE

Problema a resolver: Centro Logístico Óptimo

Una empresa cuenta con N centros logísticos conectados por rutas. Se dispone de un grafo A que indica el costo directo de transporte entre cada par de centros. Diseñar un algoritmo que determine qué centro logístico debería elegirse centro principal, de modo que el costo máximo hacia cualquier otro centro sea el mínimo.

A modo de ejemplo se tiene el siguiente caso:

Supongamos que tenemos el siguiente escenario en forma de grafo representado por matriz (A):

	1	2	3
1	0	4	10
2	∞	0	1
3	8	∞	0

El grafo de costo mínimo entre los centros directa o indirecta sería (este grafo debe calcularlo, no es dato):

	1	2	3
1	0	4	5
2	9	0	1
3	8	12	0

Centro 1 → max = 5 ; Centro 2 → max = 9 ;Centro 3 → max = 12 → el resultado sería el centro1 que es el menor valor.

¡Muchas gracias!

UADE